

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering
24 Series

Course No.: 1200

Course Title: Competitive Programming Sessional

Experiment No. : 01

Experiment Title : Solutions to 10 Codeforces Problems

<u>Submitted by:</u> Fardin Bin Aslam Numan Roll: 2403179 Section: C Session: 2024-25	<u>Submitted to:</u> Md. Mazharul Islam Lecturer Dept. of Computer Science & Engineering, RUET
--	---

Date of Experiment : 29 November 2025

Date of Submission : 13 December 2025

1. Watermelon

Problem Statement:

In this problem, we are given the weight of a watermelon in kilograms. We need to check whether it is possible to divide the watermelon into two parts such that both parts have even weights and both parts are greater than zero. If it is possible, we print YES. Otherwise, we print NO

Theory/Approach:

To solve this problem, I first observed that:

- An even number can be split into two even numbers
- But the smallest even number is 2, and 2 cannot be split into two positive even numbers

So, for the division to be possible:

- The weight must be even
- The weight must be greater than 2

If both conditions are true, the answer is YES. Otherwise, the answer is NO

Code:

```
C++ 1_Watermelon.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int w;
8      cin >> w;
9
10     if (w == 2)
11     {
12         cout << "NO" << endl;
13     }
14     else if (w % 2 == 0)
15     {
16         cout << "YES" << endl;
17     }
18     else
19     {
20         cout << "NO" << endl;
21     }
22 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 1_Watermelon.cpp -o 1_Watermelon && ./1_Watermelon
8
YES
```

Verdict:

353218761	Dec/12/2025 21:41 UTC+6	fardinnuman	4A - Watermelon	C++23 (GCC 14-64, msys2)	Accepted	62 ms	100 KB
-----------	-------------------------	-------------	-----------------	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(1)$, because no loop is used
- Space Complexity: $O(1)$, no extra memory is needed
- Insights: This problem mainly tests logical thinking, not coding difficulty

2. Bit++

Problem Statement:

We start with a variable $x = 0$. We are given several operations like $++X$, $X++$, $--X$, $X--$. We have to calculate the final value of x

Theory/Approach:

- Read each operation as a string
- The middle character of the string decides the operation:
 - For '+', we get increment
 - For '-', we get decrement

This works because the problem guarantees that the operator is always at index 1

Code:

```
C++ 2_BitPlusPlus.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, x = 0;
8      cin >> n;
9
10     while (n--)
11     {
12         string str;
13         cin >> str;
14         if (str[1] == '+')
15         {
16             x++;
17         }
18         else
19         {
20             x--;
21         }
22     }
23     cout << x;
24 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 2_BitPlusPlus.cpp -o 2_BitPlusPlus && ./2_BitPlusPlus
1
++X
1

● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 2_BitPlusPlus.cpp -o 2_BitPlusPlus && ./2_BitPlusPlus
2
X++
--X
0
```

Verdict:

353218554	Dec/12/2025 21:39 ^{UTC+6}	fardinnuman	282A - Bit++	C++23 (GCC 14-64, msys2)	Accepted	46 ms	0 KB
-----------	------------------------------------	-------------	--------------	--------------------------	----------	-------	------

Discussion:

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$
- Insights: Small observation can greatly simplify code

3. Elephant

Problem Statement:

An elephant wants to reach a point at distance n . It can move 1 to 5 steps at a time. We have to find the minimum number of moves required

Theory/Approach:

The elephant should always take the largest step possible (5). So, the minimum number of steps is: $(n + 4) / 5$. This formula ensures correct rounding up

Code:

```
C++ 3_Elephant.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9      cout << (n + 4) / 5;
10 }
```

Output:

```
• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 3_Elephant.cpp -o 3_Elephant && ./3_Elephant
5
1
• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 3_Elephant.cpp -o 3_Elephant && ./3_Elephant
12
3
```

Verdict:

353218839	Dec/12/2025 21:42 ^{UTC+6}	fardinnuman	617A - Elephant	C++23 (GCC 14-64, msys2)	Accepted	46 ms	100 KB
---------------------------	------------------------------------	-------------	---------------------------------	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(1)$
- Space Complexity: $O(1)$
- Insights: Greedy approach works best here

4. Domino Piling

Problem Statement:

We are given the dimensions of a rectangular board with n rows and m columns. Each domino covers exactly two adjacent cells. Our task is to find the maximum number of dominoes that can be placed on the board without overlapping

Theory/Approach:

- The total number of cells in the board is $n \times m$
- Since each domino covers 2 cells, the maximum number of dominoes is: $(n \times m) / 2$
- We take integer division because partial dominoes cannot be placed

This approach works because domino placement depends only on the total number of cells

Code:

```
C++ 4_DominoPiling.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, m;
8      cin >> n >> m;
9      cout << (n * m) / 2;
10 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 4_DominoPiling.cpp -o 4_DominoPiling && ./4_DominoPiling
2 4
4

● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 4_DominoPiling.cpp -o 4_DominoPiling && ./4_DominoPiling
3 3
4
```

Verdict:

353219413	Dec/12/2025 21:46 ^{UTC+6}	fardinnuman	50A - Domino piling	C++23 (GCC 14-64, msys2)	Accepted	92 ms	100 KB
---------------------------	------------------------------------	-------------	-------------------------------------	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(1)$, constant time
- Space Complexity: $O(1)$, no extra memory used
- Insights: Sometimes problems can be solved using simple math instead of simulation

5. Team

Problem Statement:

A team has three members. For each problem, each member either knows the solution (1) or does not know (0). The team will solve the problem only if at least two members know the solution. We need to count how many problems the team can solve

Theory/Approach:

For each problem:

- Take three integers
- Add them
- If the sum is greater than or equal to 2, increase the answer count

This works because when the sum is 2 or 3, it means at least two people agreed to solve the problem

Code:

```

C++ 5_Team.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, count = 0;
8      cin >> n;
9      while (n--)
10     {
11         int a, b, c;
12         cin >> a >> b >> c;
13
14         if (a + b + c >= 2)
15             count++;
16     }
17     cout << count;
18 }

```

Output:

```

• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 5_Team.cpp -o 5_Team && ./5_Team
3
1 1 0
1 1 1
1 0 0
2

• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 5_Team.cpp -o 5_Team && ./5_Team
2
1 1 0
0 1 1
2

```

Verdict:

353224718	Dec/12/2025 22:28 ^{UTC+6}	fardinnuman	231A - Team	C++23 (GCC 14-64, msys2)	Accepted	62 ms	0 KB
---------------------------	------------------------------------	-------------	-----------------------------	--------------------------	----------	-------	------

Discussion:

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$
- Insights: Summing binary values simplifies logic

6. Way Too Long Words

Problem Statement:

We are given several words. If a word has more than 10 characters, we must shorten it by keeping the first and last character and replacing the middle characters with their count. If the word has 10 or fewer characters, we print it as it is

Theory/Approach:

For each word:

- Check the length of the string
- If length > 10:
 - Print first character
 - Print number of characters between first and last
 - Print last character
- Else, print the word normally

This works because the problem only cares about shortening long words, not modifying short ones

Code:

```
C++ 6_WayTooLongWords.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9      while (n--)
10     {
11         string s;
12         cin >> s;
13         if (s.length() > 10)
14             cout << s[0] << s.length() - 2 << s.back() << endl;
15         else
16             cout << s << endl;
17     }
18 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 6_WayTooLongWords.cpp -o 6_WayTooLongWords && ./6_WayTooLongWords
4
word
word
localization
l10n
internationalization
i18n
pneumonoultramicroscopicsilicovolcanoconiosis
p43s
```

Verdict:

353225253	Dec/12/2025 22:32 ^{UTC+6}	fardinnuman	71A - Way Too Long Words	C++23 (GCC 14-64, msys2)	Accepted	46 ms	100 KB
---------------------------	------------------------------------	-------------	--	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n)$, where n is the number of words
- Space Complexity: $O(1)$ extra space
- Insights: String length checking is very useful in many problems

7. Word Capitalization

Problem Statement:

We are given a word, we need to convert only the first character to uppercase. All other characters must remain unchanged

Theory/Approach:

- Check the first character
- If it is lowercase, convert it to uppercase using ASCII values or `toupper()`

Code:

```
C++ 7_WordCapitalization.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      string word;
8      cin >> word;
9
10     if (word[0] >= 'a' && word[0] <= 'z')
11     {
12         word[0] = word[0] - 'a' + 'A';
13     }
14
15     cout << word << endl;
16 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 7_WordCapitalization.cpp -o 7_WordCapitalization && ./7_W
ordCapitalization
ApPle
ApPle

● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 7_WordCapitalization.cpp -o 7_WordCapitalization && ./7_W
ordCapitalization
konjac
Konjac
```

Verdict:

353225633	Dec/12/2025 22:36 ^{UTC+6}	fardinnuman	281A - Word Capitalization	C++23 (GCC 14-64, msys2)	Accepted	62 ms	0 KB
---------------------------	------------------------------------	-------------	--	--------------------------	----------	-------	------

Discussion:

- Time Complexity: $O(1)$
- Space Complexity: $O(1)$
- Insights: ASCII manipulation is useful in character problems

8. Boy or Girl

Problem Statement:

In this problem, we are given a username as a string. We need to determine whether the user is a boy or a girl based on the number of distinct characters in the username.

- If the number of distinct characters is even, we will print "CHAT WITH HER!"
- If the number is odd, we will print "IGNORE HIM!"

Theory/Approach:

To solve this problem:

- I used a set to store all unique characters of the username
- A set automatically removes duplicate characters
- After inserting all characters, I checked the size of the set
- If the size is even, the answer is CHAT WITH HER!
- Otherwise, the answer is IGNORE HIM!

Code:

```
C++ 8_BoyOrGirl.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      string s;
8      cin >> s;
9      set<char> letters(s.begin(), s.end());
10     cout << (letters.size() % 2 ? "IGNORE HIM!" : "CHAT WITH HER!");
11 }
```

Output:

```

• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 8_BoyOrGirl.cpp -o 8_BoyOrGirl && ./8_BoyOrGirl
  wjmzbmr
  CHAT WITH HER!

• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 8_BoyOrGirl.cpp -o 8_BoyOrGirl && ./8_BoyOrGirl
  xiaodao
  IGNORE HIM!

• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 8_BoyOrGirl.cpp -o 8_BoyOrGirl && ./8_BoyOrGirl
  sevenkplus
  CHAT WITH HER!

```

Verdict:

353226100	Dec/12/2025 22:40 ^{UTC+6}	fardinnuman	236A - Boy or Girl	C++23 (GCC 14-64, msys2)	Accepted	92 ms	100 KB
---------------------------	------------------------------------	-------------	------------------------------------	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n)$, where n is the length of the username
- Space Complexity: $O(n)$, because the set stores unique characters
- Insights: Sets are very useful when uniqueness matters

9. Petya and Strings

Problem Statement:

We are given two strings. Our task is to compare them lexicographically, but without considering lettercase

- Print -1 if the first string is smaller
- Print 1 if the first string is larger
- Print 0 if both strings are equal

Theory/Approach:

To compare the strings correctly:

- First, convert both strings to the same case (lowercase)
- Then compare the strings character by character
- Based on the comparison, print -1, 1 or 0

This works because converting both strings to lowercase removes case differences

Code:

```

C++ 9_PetyaAndStrings.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      string a, b;
8      cin >> a >> b;
9      for (char &c : a)
10         c = tolower(c);
11     for (char &c : b)
12         c = tolower(c);
13     cout << (a < b ? -1 : (a > b ? 1 : 0));
14 }

```

Output:

- fardinnuman@fardinnuman:~/Desktop/CP\$ g++ 9_PetyaAndStrings.cpp -o 9_PetyaAndStrings && ./9_PetyaAndStrings
aaaa
aaaA
0
- fardinnuman@fardinnuman:~/Desktop/CP\$ g++ 9_PetyaAndStrings.cpp -o 9_PetyaAndStrings && ./9_PetyaAndStrings
abs
Abz
-1
- fardinnuman@fardinnuman:~/Desktop/CP\$ g++ 9_PetyaAndStrings.cpp -o 9_PetyaAndStrings && ./9_PetyaAndStrings
abcdefg
AbCdEfF
1

Verdict:

353226496	Dec/12/2025 22:43 ^{UTC+6}	fardinnuman	112A - Petya and Strings	C++23 (GCC 14-64, msys2)	Accepted	62 ms	100 KB
---------------------------	------------------------------------	-------------	--	--------------------------	----------	-------	--------

Discussion:

- Time Complexity: $O(n)$, where n is the string length
- Space Complexity: $O(n)$, due to storing lowercase versions
- Insights: Case-insensitive comparison is common in real applications

10. Stones on the Table

Problem Statement:

We are given a row of stones where each stone has a color. We must remove the minimum number of stones so that no two adjacent stones have the same color

Theory/Approach:

- Loop through the string from the second character
- If the current stone color is the same as the previous one, increment a counter

We don't actually remove stones; we only count how many removals are needed.

Code:

```
C++ 10_StonesOnTheTable.cpp > ...
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n, ans = 0;
8      cin >> n;
9
10     string s;
11     cin >> s;
12
13     for (int i = 1; i < n; i++)
14         if (s[i] == s[i - 1])
15             ans++;
16
17     cout << ans;
18 }
```

Output:

```
● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 10_StonesOnTheTable.cpp -o 10_StonesOnTheTable && ./10_St
onesOnTheTable
3
RRG
1

● fardinnuman@fardinnuman:~/Desktop/CP$ g++ 10_StonesOnTheTable.cpp -o 10_StonesOnTheTable && ./10_St
onesOnTheTable
5
RRRRR
4
```

```
• fardinnuman@fardinnuman:~/Desktop/CP$ g++ 10_StonesOnTheTable.cpp -o 10_StonesOnTheTable && ./10_StonesOnTheTable
4
BRBG
0
```

Verdict:

353227477	Dec/12/2025 22:52 ^{UTC+6}	fardinnuman	266A - Stones on the Table	C++23 (GCC 14-64, msys2)	Accepted	92 ms	0 KB
---------------------------	------------------------------------	-------------	--	--------------------------	----------	-------	------

Discussion:

- Time Complexity: $O(n)$
- Space Complexity: $O(1)$
- Insights: Adjacent comparisons are common in string problems